

ФСП «спец. трек»

Цифровая платформа-агрегатор ИТ-вакансий, в которой работодатель находит кандидата и направляет приглашение, а доступ к банку профилей определяется уровнем организации процессов в компании. Документация по реализации MVP.

Назначение документа: описание архитектуры, модели данных, бизнес-логики, карты маршрутов и пользовательских сценариев MVP. Отдельная глава посвящена тому, какие служебные механизмы сейчас намеренно открыты в интерфейсе и должны быть закрыты в продуктивной версии.

Стек: Python 3.13 · FastAPI · SQLAlchemy · SQLite · Jinja2 · Pico CSS · pdfplumber

Статус: MVP для демонстрации на хакатоне ФСП. Документ — черновик; источник требований — `TZ.txt` и «ФСП спец. трек T3.pdf» в корне репозитория.

Дата: октябрь 2026 г.

Важно: приложение работает в демонстрационном режиме

Сейчас в интерфейсе видны **все внутренние логики и административные процессы**: веса и формула начисления релевантности, методика подсчёта баллов и категорий ФСП, сигналы и арифметика рейтинга компании, причина закрытия профиля, уровень организации процессов работодателя (тип), матрица видимости целиком, код подтверждения входа прямо на экране, демо-учётные записи, мок-реестр ФСП, тестовый сертификат КИТ и открытая документация API (`/api/docs`).

В продуктивной версии кандидат и работодатель этого видеть **не будут**. Полный перечень с обоснованием — **глава 10**, план закрытия — **глава 13**.

Содержание

1. Назначение платформы и ключевая механика
2. Термины и сокращения
3. Архитектура, состав модулей и стек
4. Модель данных
5. Бизнес-логика: правила, баллы и ранжирование
6. Парсер сертификата КИТ
7. Аутентификация, сессии и регистрация
8. Карта маршрутов и шаблонов
9. Пользовательские сценарии
- | 0. **Демонстрационный режим: что видно сейчас и что закрывается в продукте**
- | 1. Известные отличия реализации от ТЗ и ограничения MVP
- | 2. Запуск, проверка и демо-данные
- | 3. Чек-лист вывода в продакшн
- | 4. Приложение: справочник констант

Номера разделов в тексте соответствуют разделам `TZ.txt` (например, «раздел 6.5» — матрица видимости, «раздел 8» — парсер КИТ).

1. Назначение платформы и ключевая механика

Платформа ФСП «спец. трек» связывает амбициозных ИТ-специалистов — участников и выпускников ФСП, держателей сертификатов КИТ i.moscow — и организации с высоким уровнем организации инженерных процессов.

Инверсия привычной механики job-сайта: не кандидат рассылает отклики, а работодатель находит кандидата в банке профилей и направляет приглашение с вилкой зарплаты. Кандидат получает входящие предложения и сам решает, принимать или отклонить. Дополнительно реализован встречный поток: кандидат видит «банк потребностей» (открытые вакансии) и может откликнуться сам.

1.1. Три источника достоверности профиля

- **Сертификат КИТ i.moscow** — центр оценки компетенций: грейд, баллы (0–100), специальность, номер и дата сертификата. Загружается PDF или текстовая копия, разбирается регулярными выражениями (глава 6). Грейд из сертификата имеет высший приоритет и даёт метку «верифицирован».
- **История ФСП** — достижения участника (дисциплина, соревнование, место, роль, баллы). В MVP берётся из мок-реестра `fsp_mock.py` по email; в продукте — из реестра ФСП. Даёт сумму баллов и категорию S–D.
- **Барометр амбиций** — три вопроса анкеты кандидата, из которых выводятся четыре флага; два и более флагов дают вердикт «амбициозный профиль».

1.2. Ключевая идея разграничения доступа

Работодатель видит не всех кандидатов, а только тех, чей уровень профиля соответствует его «тиру» — оценке того, как компания относится к сотрудникам и как устроены её инженерные процессы. Тир считается из шести нейтральных вопросов анкеты (раздел 6.4 ТЗ) и определяет: (а) какие профили доступны в банке, (б) какой состав полей карточки открыт. ФИО и контакты кандидата раскрываются работодателю только после того, как кандидат принял приглашение.

1.3. Принципы реализации

- **Детерминированность.** Никаких ML-моделей, эмбедингов, векторных БД и внешних сервисов: словари, регулярные выражения, сравнения, сортировки. Каждый признак объясняется текстом.
- **Лёгкость.** Запуск на обычном ноутбуке без GPU; SQLite + FastAPI + шаблоны; отклик мгновенный (сотни записей).
- **Без брокеров и воркеров.** Нет Redis, RabbitMQ, Celery, фоновых задач и сложных миграций.
- **Производные значения не хранятся.** Грейд, амбициозность, категория ФСП, релевантность считаются на лету из правил — не разъезжаются при изменении исходных данных.

2. Термины и сокращения

Термин	Определение в рамках реализации
ФСП	Форум стратегического проектирования / соревнования, история участия в которых даёт кандидату баллы, роль и категорию. В MVP — мок-реестр.
КИТ (i.moscow/kit)	Центр оценки компетенций; выдаёт сертификат с грейдом, баллами и специальностью. Источник верификации профиля.
Тир (уровень организации процессов)	Целое число 1...3, считается из шести ответов анкеты работодателя: 5–6 «здоровых» ответов → 3, 3–4 → 2, 0–2 → 1. Для кандидата обозначается словами: «развитые» / «базовые» / «начальные процессы».
Банк кандидатов	Совокупность профилей, доступных работодателю данного тира; открытые профили с полностью заполненной анкетой.
Банк потребностей	Опубликованные вакансии (<code>Vacancy.open = True</code>), которые видит кандидат на <code>/needs</code> .
Потребность / вакансия	Запрос работодателя: специализация, грейд и режим сопоставления грейда, стек, вилка зарплаты, формат работы, город, размер команды.
Барометр амбиций	Анкета кандидата из трёх вопросов, порождающая флаги <code>captain</code> , <code>influence</code> , <code>wide_stack</code> , <code>fsp_captain</code> ; порог вердикта — 2 флага.
Категория ФСП (S–D)	Итог истории ФСП по порогам суммы баллов: $\geq 150 \rightarrow S$, $\geq 100 \rightarrow A$, $\geq 60 \rightarrow B$, $\geq 30 \rightarrow C$, иначе D.
Релевантность (match score)	Процент 0–100 соответствия пары «кандидат ↔ потребность», считается по весам признаков и сопровождается списком человеческих причин.
Приглашение (Invitation)	Основная механика: работодатель → кандидату, сообщение + вилка зарплаты, статусы <code>sent</code> → <code>viewed</code> → <code>accepted/rejected</code> .
Отклик (Application)	Встречная механика: кандидат → работодателю на потребность, те же статусы.
Верифицирован	<code>Candidate.is_verified = True</code> , если грейд подтверждён сертификатом КИТ (производное свойство, не поле БД).
Профиль закрыт (restricted)	Кандидат не виден работодателю: профиль заполнен не полностью, скрыт кандидатом из банка либо его уровень выше доступного тира работодателя.
Красные флаги	Маркеры сомнительных формулировок вакансии (<code>RED_FLAG_MARKERS</code>) и признаки неполноты профиля кандидата (<code>red_flags()</code>).

3. Архитектура, состав модулей и стек

3.1. Слои приложения

HTTP-запрос

- main.py приложение, lifespan (create_all), /static, middleware сессии → request.state.me, обработчик RedirectException (flash + 303)
- routers/*.py маршруты: проверка прав (auth.require_*), разбор формы, вызов services/helpers, render()
- services.py прикладные операции: apply_candidate_profile, connect_fsp, parse_kit_upload, apply_kit, apply_employer_survey, refresh_candidate
- helpers.py сборка данных для страниц: карточки кандидатов с фильтрацией полей по тир, банк, подборка, матрица видимости, рейтинг компании, счётчики
- logic.py детерминированные правила: специализация, грейд, амбициозность, тир, видимость, ранжирование, релевантность, справочники и форматирование
- employer_rating.py мок-рейтинг компании (сигналы + объяснение)
- fsp_mock.py мок-реестр ФСР: достижения, баллы, категории
- kit_parser.py регулярки и pdfplumber для сертификата КИТ
- auth.py Principal, коды, токены, TTL, guard-функции
- content.py тексты анкет и служебные сноски
- models.py SQLAlchemy-таблицы + производные свойства
- database.py engine, get_db, init_db, DATABASE_URL
- web.py Jinja2Templates, глобальные функции шаблонов, render(), flash/redirect, cookie-константы

3.2. Файлы и ответственность

Модуль	Раздел ТЗ	Что делает
main.py	11.1, 15.1, 15.2, 16.2	Точка входа: создание приложения, подключение роутеров, раздача /static, middleware, читающий cookie fsp_session и кладущий Principal в request.state.me, маршруты / и /health, Swagger на /api/docs.
database.py	15.1	SQLite через SQLAlchemy; путь задаётся DATABASE_URL; init_db() = SQLAlchemy.metadata.create_all (без миграций).
models.py	5, 6.8	Таблицы Candidate, Employer, Vacancy, Achievement, Invitation, Application, UserSession (имя таблицы session); константы статусов; производные свойства grade, ambition, is_verified, is_public, survey_completed.
logic.py	6, 10.3, 14.4	Вся бизнес-логика: справочники, определение специализации и грейда, амбициозность, тир, матрица видимости, состав полей по тирам, ранжирование, релевантность с объяснением, валидации, маскирование ФИО, красные флаги.
services.py	7, 8, 12	Применение форм к моделям, пересчёт производных полей, подключение/отключение истории ФСР, разбор и запись сертификата КИТ, сохранение анкеты работодателя с пересчётом тира.

Модуль	Раздел ТЗ	Что делает
seed.py	9.3, 16.2	Демо-стенд для защиты проекта: 6 кандидатов, 3 работодателя, 4 потребности и 2 приглашения, созданные штатными сервисами; автозаполнение пустой базы при старте (<code>ensure_demo_data</code>), пересборка (<code>reset_demo</code> , CLI <code>--force</code>), отчёт о стенде (<code>summary_lines</code>) и список демо-доступов для <code>/login</code> (<code>demo_accounts</code>).
helpers.py	8, 11, 13	Готовые блоки контекста для шаблонов: <code>candidate_card</code> (фильтр полей по тире смотрящего), <code>candidate_full</code> , <code>fsp_block</code> , <code>kit_block</code> , <code>bank_rows</code> , <code>vacancy_candidates</code> , <code>visibility_matrix</code> , <code>match_rows</code> , <code>company_rating</code> , <code>public_stats</code> .
auth.py	12	Генерация кода (6 цифр) и токена (<code>secrets.token_urlsafe(24)</code>), TTL сессии 7 дней и кода 15 минут, класс <code>Principal</code> с ролями и URL, guard-функции <code>require_candidate</code> , <code>require_employer</code> , <code>require_new_user</code> .
kit_parser.py	8	Извлечение текста PDF (pdfplumber) и разбор полей сертификата регулярными выражениями; при любой ошибке — <code>None</code> , приложение не падает; содержит <code>DEMO_CERTIFICATE_TEXT</code> для демо-загрузки.
fsp_mock.py	9	Мок-реестр ФСЦП: 8 тестовых участников, сумма баллов достижений, пороги категорий, старшая роль, список дисциплин.
employer_rating.py	14.1	Мок-рейтинг компании 0–10: сигналы (отзывы, реестр отечественного ПО, тендеры, разрыв зарплат) + объяснение; для недемо-компаний значения детерминированно вычисляются из MD5-хеша названия.
content.py	7, 9.4, 14.1	Формулировки шести вопросов анкеты работодателя, трёх вопросов барометра, демонстрационные сноски (<code>FSP_DEMO_NOTE</code> , <code>AUTH_DEMO_NOTE</code> , <code>RATING_DEMO_NOTE</code>), шаги регистрации.
web.py	11, 12.5	Окружение Jinja2 и глобальные функции/справочники шаблонов, <code>render()</code> с автоподмешиванием flash, <code>me</code> , заголовка и значений по умолчанию <code>errors=[]</code> , <code>error=None</code> ; <code>RedirectException</code> → flash-cookie + 303.
routers/	10	<code>auth.py</code> (вход, код, выход), <code>registration.py</code> (пошаговые анкеты), <code>candidate.py</code> (кабинет, потребности, отклики, приглашения), <code>employer.py</code> (кабинет, анкета, вакансии, банк, приглашения, отклики).
templates/	11	30 шаблонов Jinja2: <code>base.html</code> , лендинг, вход и код подтверждения, 5 страниц регистрации, страницы кабинета кандидата, страницы кабинета работодателя, 6 переиспользуемых парциалов карточек и форм.
static/css/	11.1	<code>pico.min.css</code> + <code>app.css</code> (карточки, бейджи, шаги регистрации, флеш-сообщения, сетка).
tests/	12.3, 12.4	Набор pytest (95 проверок): <code>conftest.py</code> (временная БД через <code>DATABASE_URL</code> , сид-данные, токены ролей, <code>TestClient</code> без прохождения редиректов), <code>test_smoke_routes.py</code> (компиляция 30 шаблонов, обход всех GET-путей под тремя ролями, guard-функции и изоляция ролей), <code>test_smoke_strict.py</code> (рендер всех страниц при <code>StrictUndefined</code>), <code>test_smoke_employer_flow.py</code> (анкета → тир → банк → потребность → приглашение → контакты, КИТ, матрица видимости), <code>test_seed.py</code> (демо-стенд: состав, матрица видимости, тир, ранжирование, идемпотентность, <code>--force</code>), <code>test_kit_specialization.py</code> (специализация по полю специальности сертификата КИТ).

3.3. Соглашения кода

- Каждая функция помечена номером раздела ТЗ в docstring — соответствие требованиям проверяется построчно.
- Все страницы рендерятся на сервере (Jinja2); формы отправляются POST и завершаются редиректом 303 с одноразовым flash-сообщением в cookie.
- Маршруты не фильтруют поля кандидатов самостоятельно: разграничение по тиру всегда проходит через `helpers.candidate_card()` → `logic.candidate_visible_profile()`.

- Проверка прав — guard-функции `auth.py`: при отсутствии доступа бросается `RedirectException`, её перехватывает общий обработчик в `main.py`.
- Шаблоны обращаются к словарям как к элементам (`row['match']`), а не как к атрибутам (`row.match`), чтобы не попадать на `dict.items()`; глобальная функция `tier_label` продублирована алиасом `tier_title`, потому что маршруты работодателя кладут в контекст одноимённую строку.

3.4. Зависимости

```
fastapi==0.142.2      uvicorn[standard]==0.54.0  jinja2==3.1.6
sqlmodel==0.0.47      python-multipart==0.0.20  pdfplumber==0.11.10
# только для локального прогона тестов:
pytest==9.1.1         httpx==0.28.1
```

4. Модель данных

4.1. Состав БД

Таблица	Класс	Назначение
candidate	Candidate	Профиль кандидата: анкета, барометр амбиций, подключение ФСП, данные сертификата КИТ, служебные флаги.
employer	Employer	Профиль организации и ответы анкеты процессов (JSON), из которых считается тир.
vacancy	Vacancy	Потребность работодателя: специализация, грейд и режим сопоставления, стек, вилка, формат, город, команда.
achievement	Achievement	Достижение ФСП (дисциплина, соревнование, место, роль, баллы). Создана по ТЗ 5.4; в MVP заполняется из мок-реестра по email, а не из этой таблицы (см. главу 11).
invitation	Invitation	Приглашение работодателя кандидату: сообщение, вилка, статус.
application	Application	Отклик кандидата на потребность: сообщение, статус.
session	UserSession	Сессия и незавершённая регистрация: токен, роль, код подтверждения, срок действия, признак подтверждения email.

4.2. Candidate — кандидат

Поле	Тип	Пояснение
id	int, PK	Первичный ключ.
name, email	str	ФИО и логин; email уникальный (индекс). До принятия приглашения работодателю показывается маскированное имя.
city	str	Город; виден работодателю только tier 3.
specialization	str	Код специализации (backend, frontend, ds, devops, qa, support, security, mobile, embedded, other).
specialization_manual	bool	Служебное: выбрано вручную — автоопределение больше не перезаписывает значение.
team_role, work_format	str	Ответы барометра: желаемая роль и формат работы.
stack	JSON list	Стек; участвует в определении специализации и флага «широкий стек» (3+ направлений).
fsp_role	str	Роль в команде ФСП (капитан / архитектор / разработчик / участник); источник флага fsp_captain.
ambitions	JSON list	Коды сработавших флагов барометра.
grade_override	str	Служебное: ручной/вычисленный грейд, если КИТ не загружен.
fsp_connected, fsp_demo	bool	Подключена ли история ФСП; fsp_demo — служебное: данные пришли из мок-реестра.
fsp_score, fsp_category	int, str	Сумма баллов достижений и категория S-D (по умолчанию 0 и D — «правило пустого ФСП»).
kit_connected, kit_certificate_number, kit_competition, kit_place, kit_team, kit_grade, kit_score, kit_date	разные	Поля сертификата КИТ: номер, соревнование, место, состав команды, грейд, баллы 0–100, дата выдачи.
kit_raw, kit_source	str	Служебные: распознанный текст сертификата и источник (pdf / text).
consent	bool	Согласие на обработку персональных данных (152-ФЗ).
visible_in_bank	bool	Видимость профиля в банке; кандидат может скрыть себя.
created_at, updated_at	str	Метки времени в формате ISO-8601 UTC.

Производные свойства (не хранятся в БД)

`grade` — КИТ → ручной выбор → барометр → junior; `ambition` — сумма флагов барометра ≥ 2 ; `is_verified` — `kit_connected` and (`kit_grade` or `kit_certificate_number`); `is_public` — синоним `visible_in_bank`.

4.3. Employer — работодатель

Поле	Тип	Пояснение
id	int, PK	Первичный ключ.
company, email	str	Название организации и логин (email уникальный).
contact_name, position, city, description	str	Контактное лицо, должность, город, описание деятельности.
survey_answers	JSON dict	Шесть ответов анкеты: <code>q1_lead_time</code> , <code>q2_support</code> , <code>q3_remote</code> , <code>q4_mistake</code> , <code>q5_outsource</code> , <code>q6_benefits</code> ; вариант A = <code>true</code> .
tier	int	Уровень организации процессов 1–3. Пересчитывается при каждом сохранении анкеты (<code>services.apply_employer_survey</code>). Отличие от ТЗ 5.2: там тир не хранится в БД — см. главу 11.
consent	bool	Согласие на обработку данных.
last_visit_at	str	<i>Служебное:</i> для счётчика «новых откликов» на дашборде.
created_at, updated_at	str	Метки времени ISO-8601 UTC.

Свойство `survey_completed` — анкета заполнена, если даны все 6 ответов. По ТЗ работодатель не должен видеть свой тир; в демо-сборке он показывается словами — см. главу 10, п. 5.

4.4. Vacancy — потребность

Поле	Тип	Пояснение
id, employer_id	int	Ключ и внешний ключ на работодателя.
company	str	<i>Служебное:</i> копия названия компании для карточек (чтобы не делать лишний JOIN).
title, description	str	Название позиции и текст потребности; текст проверяется на маркеры «красных флагов».
specialization, grade	str	Категория и требуемый грейд.
grade_match	str	Режим сопоставления грейда: <code>any</code> / <code>exact</code> / <code>plus_minus</code> (по умолчанию <code>plus_minus</code>) — расширение раздела 10.3 ТЗ.
stack	JSON list	Требуемый стек; доля совпадений даёт баллы релевантности.
salary_min, salary_max	int	Вилка зарплаты обязательна: нули не проходят валидацию (<code>logic.validate_salary</code>).
format, city, team_size	str	<code>remote</code> / <code>office</code> / <code>hybrid</code> , город, размер команды.
open, is_new	bool	<i>Служебные:</i> публикация в банке потребностей и отметка «новая» для кандидата.
created_at, updated_at	str	Метки времени.

4.5. Связи и статусы

Таблица	Ключевые поля	Правила
achievement	candidate_id, discipline, competition, place, role, points, date	Источник баллов ФСП. Сумма <code>points</code> → <code>fsp_score</code> ; старшая роль → <code>fsp_role</code> .
invitation	employer_id, candidate_id, vacancy_id, message, salary_min/max, status, is_new	Вилка обязательна. Статусы: <code>sent</code> → <code>viewed</code> → <code>accepted</code> / <code>rejected</code> . Контакты кандидата раскрываются работодателю только при <code>accepted</code> .

Таблица	Ключевые поля	Правила
application	candidate_id, employer_id, vacancy_id, message, status, is_new	Отклик кандидата; те же статусы. Работодатель меняет статус через <code>POST /employer/applications/{id}/status</code> .
session	token, user_id, email, role, code, code_expires_at, is_verified	До подтверждения кода хранит «незавершённую регистрацию»: <code>user_id</code> пустой, есть email, роль и код. TTL сессии 7 дней, TTL кода 15 минут.

5. Бизнес-логика: правила, баллы и ранжирование

Все правила сосредоточены в `logic.py` и детерминированы: словари, регулярные выражения, сравнения, арифметика. Ниже — действующие формулы.

5.1. Специализация кандидата (раздел 6.1)

Приоритет источников: **1) КИТ-сертификат** → **2) дисциплина ФСП** → **3) стек** → **4) ручной выбор**.

Автоопределение не перезаписывает значение, если кандидат выбрал специализацию вручную (`specialization_manual = True`).

- КИТ:** текст специальности из сертификата сопоставляется со словарём `KIT_SPECIALTY_KEYWORDS` (например, «технической поддержки» → `support`, «данным»/«машинн» → `ds`, «фронт»/«веб» → `frontend`, «разработ»/«python» → `backend`).
- ФСП:** дисциплины достижений сопоставляются с `FSP_DISCIPLINE_KEYWORDS` («алгоритм» → `backend`, «беспилот»/«робот» → `embedded`, «данн»/«ml»/«ai» → `ds`, «продуктов» → `backend`).
- Стек:** `STACK_SPEC_KEYWORDS`; порядок словаря задаёт приоритет при равенстве числа совпадений.

Специальность берётся из распознанного поля сертификата (`logic.kit_specialty_text` → `kit_parser.parse_kit_text`), а не из всего текста документа: в шапке любого сертификата есть строка «Прошел(а) тестирование по специальности ...» (п. 8.1 ТЗ), поэтому поиск ключевых слов по всему `kit_raw` срабатывал на слове «тестирование» и уводил любую анкету в QA. Регрессию проверяет `tests/test_kit_specialization.py`.

Коды специализаций: `backend`, `frontend`, `ds`, `devops`, `qa`, `support`, `security`, `mobile`, `embedded`, `other` (словарь `SPECIALIZATIONS`).

5.2. Грейд кандидата (раздел 6.2)

```
resolve_grade(kit_grade, kit_certificate_number, ambitions, grade_override):
    1. kit_grade из сертификата          → источник "kit"          (verified = True)
    2. буква J/M/S в номере сертификата → источник "kit_number" (verified = True)
       формат номера: ГГГГММДД-Н-{J|M|S}-ННН
    3. grade_override (ручной выбор)      → источник "manual"      (verified = False)
    4. барометр: ≥ 2 флагов               → middle, "barometer"    (verified = False)
    5. иначе                             → junior, "default"      (verified = False)
```

Верифицированным считается только грейд из сертификата КИТ: он влияет на матрицу видимости, на ранжирование и даёт баллы в релевантности.

5.3. Барометр амбиций (разделы 6.3 и 7.3)

Код флага	Условие	Формулировка для кандидата
<code>captain</code>	<code>team_role</code> ∈ {капитан, архитектор}	роль в команде — капитан или архитектор
<code>influence</code>	<code>work_format</code> = «обсуждаем и влияю»	формат работы — «обсуждаем и влияю»
<code>wide_stack</code>	стек ≥ 3 элементов (<code>is_wide_stack</code>)	широкий стек (3 и более направлений)
<code>fsp_captain</code>	<code>fsp_role</code> ∈ {капитан, архитектор}	роль в команде ФСП — капитан или архитектор

Вердикт: `ambition = True`, если сработало ≥ 2 флагов (`AMBITION_THRESHOLD = 2`). Текстовая расшифровка (`ambitions_note`) сообщает количество флагов, порог, список сработавших и вердикт — в демо-сборке она показывается кандидату и на лендинге (см. главу 10, п. 17).

5.4. Тир работодателя (разделы 6.4 и 7.1)

Шесть нейтральных вопросов анкеты, вариант А = «здоровый» ответ = `True`. `tier_score` = число истинных ответов (0–6), далее:

tier_score ≥ 5 → тип 3 «развитые процессы»
 tier_score 3–4 → тип 2 «базовые процессы»
 tier_score 0–2 → тип 1 «начальные процессы»

№	Вопрос (сокращённо)	Вариант А → True	Вариант Б → False
1	Время от готовности задачи до тестового стенда	меньше рабочего дня	три дня и больше
2	Решение бытовых/периферийных проблем сотрудника	один запрос в поддержку, меняют в течение дня	ищет сам, договаривается с АХО, ждёт
3	Рабочее место удалённого сотрудника	портативный ноутбук, работа откуда удобно	стационарный моноблок или VPN с привязкой к месту
4	Реакция на падение тестового контура новичком	разбор на ретро, правки в онбординге	публичный разбор с последствиями
5	Внешние команды в разработке продукта	ноль или одна	несколько, часть на подряде
6	Бюджет на инженера (питание, спорт, медицина)	от 8 000 ₽	до 2 000 ₽ либо строки нет

Все шесть вопросов обязательны: вариант «не знаю» не предусмотрен, иначе тип невозможно посчитать. Формулировки намеренно нейтральные и не подсказывают «правильный» ответ (требование раздела 7.1 ТЗ).

5.5. Матрица видимости (раздел 6.5)

Минимальный тип работодателя, который видит кандидата (`candidate_min_tier`): кандидат виден, если `tier  $\geq$  min_tier`.

Строка матрицы (<code>profile_level</code>)	Мин. тип	Условие
топ ФСП	3	грейд senior и (категория ФСП S или A или роль капитан/архитектор при <code>fsp_score > 0</code>)
опытный специалист / мидл	2	грейд middle или senior (не топ ФСП)
новичок с амбициями	2	грейд junior и (вердикт «амбициозный» или роль ФСП капитан/архитектор)
новичок без амбиций	1	грейд junior без амбиций и капитанской роли — виден всем

Дополнительные фильтры банка (`visibility_allows`): профиль заполнен полностью, кандидат не скрыл себя из банка (`is_public`), матрица пройдена. Закрытые профили не исчезают бесследно: работодатель видит их число и замаскированные имена — в продукте это поведение меняется (глава 10, п. 7).

5.6. Состав полей карточки по тирам (`TIER_FIELDS`)

Тип	Поля карточки, доступные работодателю
1 — начальные процессы	специализация, грейд, метка верификации, баллы КИТ, стек, достижения ФСП кратко
2 — базовые процессы	+ амбициозность, роль в команде ФСП
3 — развитые процессы	+ город, полный профиль (достижения ФСП полностью)

Для тира 1 амбициозность не показывается вовсе (`is_ambitious = None`, `ambition_hidden = True`) — работодатель не может вывести её косвенно. ФИО и email лежат в карточке в ключах `real_name` / `email` и используются шаблонами только для приглашений со статусом `accepted`.

5.7. Релевантность пары «кандидат ↔ потребность» (разделы 6.6 и 10.3)

Функция `logic.score_match()` возвращает процент 0–100, краткую сводку и список причин. Веса (`MATCH_WEIGHTS`, сумма = 100):

Признак	Вес	Как начисляются баллы
Специализация	35	35 — если специализация кандидата совпадает со специализацией потребности; 0 — если нет (совпадение проверяет <code>specialization_match</code>).
Стек	25	$\text{round}(25 \times \text{matched} / \text{total})$ — доля совпавших элементов стека потребности. Если стек в потребности не указан: ≥ 3 элементов у кандидата $\rightarrow 25$, 1–2 элемента $\rightarrow 12$ ($25 // 2$), пусто $\rightarrow 0$.
Грейд	20	20 — грейд не задан в потребности или режим <code>any</code> ; 20 — грейд подходит по режиму (<code>exact</code> / <code>plus_minus</code>); 10 ($20 // 2$) — грейд соседний при <code>plus_minus</code> ; 0 — не совпадает.
Верификация грейда	10	10 — грейд подтверждён сертификатом КИТ; 3 ($10 // 3$) — грейд заявлен кандидатом самостоятельно.
История ФСП	10	По категории: S $\rightarrow 10$, A $\rightarrow 9$, B $\rightarrow 7$, C $\rightarrow 5$, D $\rightarrow 2$ (<code>MATCH_FSP_POINTS</code>); +1 балл за вердикт «амбициозный профиль», но не больше 10.

```
score = spec_points + stack_points + grade_points + verified_points + fsp_points
score = max(0, min(100, score))          # отсечение до диапазона 0...100
```

К каждому начислению формируется текстовая причина, например: «специализация совпадает: Backend-разработка», «стек: совпало 3 из 5 (python, fastapi, docker)», «грейд соседний: у кандидата Middle, в потребности Senior (указанный грейд ± 1 уровень)», «история ФСП: 120 баллов, категория A, амбициозный профиль».

Порядок подбора (`rank_candidates`)

1. Отсев: профиль неполный $\rightarrow$ исключён; кандидат скрыл профиль $\rightarrow$ исключён; `tier` работодателя ниже минимального тира кандидата $\rightarrow$ исключён; специализация не совпадает со специализацией потребности $\rightarrow$ исключён.
2. Сортировка по убыванию релевантности.
3. При равной релевантности — строгий порядок раздела 6.6 (`rank_key`): верифицированный грейд выше неверифицированного $\rightarrow$ баллы КИТ по убыванию $\rightarrow$ баллы ФСП по убыванию $\rightarrow$ роль ФСП капитан/архитектор выше прочих $\rightarrow$ амбициозный профиль выше.

Дашборд работодателя сортирует отклики так: сначала новые (`is_new`), затем по убыванию релевантности.

Демонстрационная прозрачность

Веса, формула и поштучные причины сейчас показываются работодателю прямо в интерфейсе (страница потребности, карточка соответствия). В продукте эта методика закрывается — подробнее в главе 10, пп. 1–2.

5.8. История ФСП: баллы и категории (раздел 9)

```
fsp_score    =  $\sum$  points по всем достижениям участника
fsp_category = S при fsp_score  $\geq 150$  | A  $\geq 100$  | B  $\geq 60$  | C  $\geq 30$  | иначе D
fsp_role     = старшая роль достижений: капитан > архитектор > разработчик > участник
specialization (источник №2) = по ключевым словам дисциплин
```

Источник данных в MVP — мок-реестр `fsp_mock.FSP_REGISTRY`: 8 тестовых участников с разными сценариями (топ ФСП, крепкий мидл, новичок-капитан с бронзой, новичок без достижений, кандидат без истории ФСП). Если email не найден — возвращается пустая история, и срабатывает «правило пустого ФСП»: `fsp_score = 0`, `fsp_category = D`, профиль остаётся работоспособным. Подключение и отключение истории — действия кандидата (`POST /candidate/fsp/connect`, `/disconnect`).

Словесные расшифровки категорий (`FSP_CATEGORY_TITLES`), например «S — топ-уровень: стабильные победы в чемпионатах», «D — история ФСП не найдена», по ТЗ видны только кандидату.

5.9. Рейтинг компании (раздел 14.1, `employer_rating.py`)

```
score = 3.0
+ (reviews - 3.0) * 1.5 # отзывы сотрудников, 0...5
+ (1.0, если компания в реестре отчет. ПО)
+ min(tenders, 20) / 20 * 1.5 # публичные закупки за год
- (salary_ratio - 5.0) * 0.6, если ratio > 5 # разрыв зарплат топ-менеджмента
+ (tier - 2) * 0.8, если тир известен # анкета процессов
score = round(clamp(score, 0, 10), 1)
label = «высокий» ≥ 7.5 | «средний» ≥ 5.0 | «низкий» иначе
```

Для четырёх демо-компаний сигналы заданы явно (ТехКорп, Вектор Софт, ПромТех Групп, ОлдЛайн), для остальных — детерминированно вычисляются из MD5-хеша названия. Рейтинг видит только кандидат: в карточке потребности, на странице потребности и в приглашении. Если тир неизвестен (карточка до приглашения), сигнал анкеты в рейтинг не добавляется — цифра тира не раскрывается никому.

5.10. Маркеры сомнительных вакансий (раздел 14.4)

Словарь `RED_FLAG_MARKERS` (13 формулировок: «молодой дружный коллектив», «как семья», «стрессоустойчив», «ненормированный рабочий день», «готовность к переработкам», «зарплата по итогам собеседования», «оформление по желанию» и др.) сканируется в описании потребности функцией `scan_markers()`; найденные маркеры показываются кандидату как предупреждение. Отдельная функция `red_flags(candidate)` собирает признаки риска в профиле кандидата: грейд не подтверждён КИТ, профиль заполнен не полностью, профиль скрыт, история ФСП не найдена (0 баллов, категория D), меньше двух флагов барометра, специализация не определена.

5.11. Анонимизация и раскрытие контактов (раздел 6.7)

- До принятия приглашения работодатель видит маскированное имя (`mask_name`): «Бурцева Вероника» → «Б. В.», одно слово → «Б••••».
- Единственное правило раскрытия: ФИО и email становятся доступны работодателю после `status = accepted` (`CONTACT_REVEAL_NOTE`).
- Email кандидата не показывается в банке и в подборке ни при каком тире.

5.12. Валидации

Проверка	Правило и сообщение
Вилка зарплаты	Обе границы обязательны и больше нуля; нижняя не больше верхней. Сообщения: «Укажите вилку зарплаты: обе границы обязательны и больше нуля», «Нижняя граница вилки не может быть больше верхней».
Email	Регулярное выражение <code>^[^@\\s]+@[^@\\s]+\\. [^@\\s]+</code> ; иначе «Укажите корректный email». Email — логин, уникальный в рамках роли.
Полнота профиля кандидата	Обязательны: имя и фамилия, специализация, роль в команде, формат работы, согласие на обработку данных (152-ФЗ). Неполный профиль не участвует в банке, ранжировании и приглашениях.
Анкета работодателя	Все 6 вопросов обязательны, допустимые значения — <code>true / false</code> ; иначе «Ответьте на вопрос: ...».
Сертификат КИТ	Если файл не похож на сертификат — «файл не похож на сертификат КИТ — заполните данные вручную в кабинете»; приложение не падает (раздел 8.4).

6. Парсер сертификата КИТ

Модули `kit_parser.py` (разбор) и `services.parse_kit_upload` (приём файла). Главное правило раздела 8.4 ТЗ: при любой ошибке возвращается `None`, кандидату предлагается ручной ввод, приложение не падает.

6.1. Что извлекается

Поле	Регулярное выражение	Пример из сертификата
Баллы (<code>kit_score</code>)	Результат\s+(\d+)\s*балл	«Результат 84 баллов из 100» → 84 (отсекается до 0...100)
Грейд (<code>kit_grade</code>)	Уровень\s+Грейда\s+([A-Za-zA-Яa-яЁё]+)	«Уровень Грейда Senior» → senior
Номер (<code>kit_number</code>)	(\d{8}-\d+-[JMSjms])-\d+	«20260119-37-S-102905»
Дата (<code>kit_date</code>)	Выдан\s+(\d{2}\.\d{2}\.\d{4})	«Выдан 19.01.2026» → 19.01.2026
Специальность	по\s+специальности\s+(.+)	«Специалист технической поддержки (L1/L2/L3)» → support
ФИО	строка после заголовка «КИТ», 2–4 слова с заглавной буквы	«Бурцева Вероника Львовна»

6.2. Порядок обработки

- PDF → текст через `pdfplumber` (`extract_pdf_text`); остальные файлы читаются как UTF-8 текст с игнорированием ошибок.
- Текст → поля (`parse_kit_text`). Если грейд словом не распознан, применяется резервное правило: буква `J/M/S` в номере сертификата (`_grade_from_number`).
- Если не распознано ни одного поля — `None`; маршрут показывает ошибку и предлагает ручной ввод.
- Успех → `services.apply_kit()`: записываются `kit_connected`, `kit_source` (pdf/text), номер, соревнование («КИТ i.moscow · специальность»), грейд, баллы, дата, распознанный текст (`kit_raw`); специализация обновляется, только если она не выбрана вручную; затем `refresh_candidate()` пересчитывает грейд и производные.
- Место и состав команды в сертификате КИТ не печатаются — поля `kit_place` и `kit_team` остаются пустыми.

В модуле есть `DEMO_CERTIFICATE_TEXT` — пример реального сертификата из раздела 8.1 ТЗ, используется для демо-загрузки и проверок. В продукте его и поле `kit_raw` в интерфейсе не показывают (глава 10, п. 11).

7. Аутентификация, сессии и регистрация

7.1. Вход без пароля

Параметр	Значение
Логин	email + роль (candidate / employer); пароль не используется
Код подтверждения	6 цифр, <code>secrets.randbelow(1_000_000)</code> , TTL 15 минут (<code>CODE_TTL_MINUTES</code>)
Токен сессии	<code>secrets.token_urlsafe(24)</code> , TTL 7 дней (<code>SESSION_TTL_DAYS</code>)
Cookie	<code>fsp_session</code> (httpOnly, sameSite=lax) и <code>fsp_flash</code> — base64-JSON одноразового сообщения, max-age 120 c
Хранение сессий	Таблица <code>session</code> в SQLite; JWT не используется
Редиректы	После POST — 303 See Other; сообщение передаётся через flash-cookie

7.2. Principal и защита маршрутов

Middleware в `main.py` читает cookie и кладёт объект `auth.Principal` в `request.state.me`. Свойства: `role`, `is_authenticated` (email подтверждён и сессия не истекла), `user` (кандидат или работодатель), `title`, `home_url` (`/candidate` или `/employer`), `register_url`, `needs_registration` (сессия подтверждена, но анкета не заполнена).

Guard-функции: `require_principal(request, role)`, `require_candidate`, `require_employer`, `require_new_user`. При нарушении бросается `RedirectException` — общий обработчик возвращает 303 и flash-сообщение, поэтому «чужой» URL всегда безопасно редиректит, а не показывает ошибку 500.

7.3. Пошаговая регистрация

Роль	Шаги	Особенности
Кандидат	1) Профиль и барометр → 2) История ФСП → 3) Сертификат КИТ	До подтверждения кода сессия хранит «незавершённую регистрацию»: <code>user_id</code> пустой, есть email, роль и код. Шаги 2 и 3 можно пропустить и пройти позже в кабинете.
Работодатель	1) Профиль компании → 2) Анкета процессов (6 вопросов)	После второго шага сохраняется <code>survey_answers</code> и пересчитывается <code>tier</code> ; без заполненной анкеты банк кандидатов недоступен.

Демонстрационный режим аутентификации

SMTP не подключён: код подтверждения возвращается в шаблон и показывается на странице `/verify` (`demo_code`), а в подвале всех страниц выводится `AUTH_DEMO_NOTE`. В продукте — ФСП ID через Keycloak или magic link на email, показ кода исключён (глава 10, п. 9).

8. Карта маршрутов и шаблонов

Всего 48 HTTP-операций (24 уникальных пути GET/POST + служебные). Все страницы серверные; API-эндпоинтов с JSON, кроме `/health` и OpenAPI, нет.

8.1. Публичные маршруты и вход

Метод	Путь	Назначение / шаблон
GET	/	Лендинг: счётчики банка, описание механики, матрица тиров. Авторизованных сразу редиректит в кабинет или на анкету. Шаблон <code>index.html</code> .
GET	/health	Техническая проверка стенда: <code>{"status": "ok"}</code> .
GET/POST	/login	Вход по email и роли; POST создаёт сессию с кодом. Шаблоны <code>login.html</code> , далее <code>verify.html</code> .
GET/POST	/verify	Подтверждение кода (6 цифр, 15 минут). В демо-режиме код показывается на странице.
POST	/logout	Завершение сессии, удаление cookie.
GET/POST	/register/candidate	Шаг 1: профиль и барометр амбиций. Шаблон <code>register_candidate_profile.html</code> .
GET/POST	/register/candidate/fsp	Шаг 2: подключение истории ФСП (в демо — список email мок-реестра). Шаблон <code>register_candidate_fsp.html</code> .
GET/POST	/register/candidate/kit	Шаг 3: загрузка сертификата КИТ (PDF/TXT), можно пропустить. Шаблон <code>register_candidate_kit.html</code> .
GET/POST	/register/employer	Шаг 1: профиль компании. Шаблон <code>register_employer_profile.html</code> .
GET/POST	/register/employer/survey	Шаг 2: анкета из 6 вопросов → тир. Шаблон <code>register_employer_survey.html</code> .

8.2. Кабинет кандидата

Метод	Путь	Назначение / шаблон
GET	/candidate	Дашборд: профиль, грейд и источник, барометр, ФСП-блок, КИТ-блок, входящие приглашения, матрица видимости. <code>candidate_home.html</code> .
GET/POST	/candidate/edit	Редактирование анкеты. <code>candidate_edit.html</code> + <code>_partial_profile_form.html</code> .
POST	/candidate/visibility	Показать/скрыть профиль в банке.
POST	/candidate/fsp/connect /candidate/fsp/disconnect	Подключить и отключить историю ФСП (пересчёт баллов и категории).
POST	/candidate/kit /candidate/kit/remove	Загрузить/заменить сертификат КИТ и удалить его.
GET	/needs	Банк потребностей: открытые вакансии с вилкой и рейтингом компании. <code>needs.html</code> .
GET	/needs/{vacancy_id}	Страница потребности: описание, рейтинг, красные флаги, форма отклика. <code>need_detail.html</code> .
POST	/needs/{vacancy_id}/apply	Отклик кандидата (сообщение) → <code>Application</code> со статусом <code>sent</code> .
GET	/candidate/applications	Мои отклики и их статусы. <code>candidate_applications.html</code> .
GET	/candidate/invitations	Входящие приглашения. <code>candidate_invitations.html</code> .
GET	/candidate/invitations/{id}	Приглашение: вилка, уровень процессов компании словами, рейтинг. <code>candidate_invitation_detail.html</code> .
POST	/candidate/invitations/{id}/accept .../reject	Принять или отклонить; при принятии работодателю раскрываются ФИО и email.

8.3. Кабинет работодателя

Метод	Путь	Назначение / шаблон
GET	/employer	Дашборд: уровень процессов, потребности, отклики (новые сверху), приглашения, размер банка. <code>employer_home.html</code> .
GET/POST	/employer/survey	Повторное прохождение анкеты процессов с пересчётом тира. <code>employer_survey_edit.html</code> + <code>_partial_survey_form.html</code> .
GET/POST	/employer/vacancies/new	Создание потребности: специализация, грейд и режим сопоставления, стек, вилка, формат, город, команда. <code>vacancy_form.html</code> .
GET	/employer/vacancies/{id}	Страница потребности: ранжированная подборка кандидатов, число и имена закрытых профилей, форма приглашения. <code>vacancy_detail.html</code> .
POST	/employer/vacancies/{id}/toggle .../delete	Снять с публикации / опубликовать, удалить потребность.
POST	/employer/vacancies/{id}/invite	Приглашение: сообщение + вилка (валидируется) → <code>Invitation</code> со статусом <code>sent</code> .
GET	/employer/bank?spec=	Банк кандидатов с фильтром по специализации: открытые карточки, счётчик закрытых, состав доступных полей. <code>employer_bank.html</code> .
GET	/employer/candidates/{id}	Карточка кандидата: открытая (<code>employer_candidate_detail.html</code>) или закрытая (<code>employer_candidate_restricted.html</code>).
GET	/employer/applications	Отклики кандидатов с релевантностью. <code>employer_applications.html</code> .
POST	/employer/applications/{id}/status	Смена статуса отклика: <code>viewed</code> / <code>accepted</code> / <code>rejected</code> .
GET	/employer/invitations	Отправленные приглашения и их статусы; при <code>accepted</code> — раскрытые ФИО и email. <code>employer_invitations.html</code> .
POST	/employer/invitations/{id}/status	Смена статуса приглашения работодателем (например, отозвать).

8.4. Шаблоны (30 файлов)

Группа	Файлы
Каркас и публичные	<code>base.html</code> , <code>index.html</code> , <code>login.html</code> , <code>verify.html</code>
Регистрация кандидата	<code>register_candidate_profile.html</code> , <code>register_candidate_fsp.html</code> , <code>register_candidate_kit.html</code>
Регистрация работодателя	<code>register_employer_profile.html</code> , <code>register_employer_survey.html</code>
Кабинет кандидата	<code>candidate_home.html</code> , <code>candidate_edit.html</code> , <code>needs.html</code> , <code>need_detail.html</code> , <code>candidate_applications.html</code> , <code>candidate_invitations.html</code> , <code>candidate_invitation_detail.html</code>
Кабинет работодателя	<code>employer_home.html</code> , <code>employer_survey_edit.html</code> , <code>vacancy_form.html</code> , <code>vacancy_detail.html</code> , <code>employer_bank.html</code> , <code>employer_candidate_detail.html</code> , <code>employer_candidate_restricted.html</code> , <code>employer_applications.html</code> , <code>employer_invitations.html</code>
Парциалы	<code>_partial_candidate_card.html</code> , <code>_partial_vacancy_card.html</code> , <code>_partial_match.html</code> , <code>_partial_profile_form.html</code> , <code>_partial_survey_form.html</code>

Глобальные переменные шаблонов (web.py)

`format_salary` · `specialization_title` · `grade_title` · `category_title` · `tier_label` · `tier_title` · `profile_level` · `status_title` · `format_title` · `money` · `percent` · `place_text` · `salary_note` · `mask_name` · `ambitions_labels` · `grade_match_note` · `stack_note` · `restricted_reason` · `SPECIALIZATIONS` · `GRADES` · `TEAM_ROLES` · `FSP_ROLES` · `WORK_FORMATS` · `VACANCY_FORMATS` · `AMBITIONS` · `GRADE_MATCH_MODES` · `TIER_FIELDS` · `TIER_DESCRIPTIONS` · `FSP_CATEGORY_TITLES` · `MATCH_WEIGHTS` · `CONTACT_REVEAL_NOTE` · `EMPLOYER_SURVEY` · `CANDIDATE_BAROMETER` · `FSP_DEMO_NOTE` · `AUTH_DEMO_NOTE` · `RATING_DEMO_NOTE` · `CANDIDATE_STEPS` · `EMPLOYER_STEPS`

Почему это важно для главы 10

Справочники `MATCH_WEIGHTS`, `TIER_FIELDS`, `TIER_DESCRIPTIONS` и сноски `*_DEMO_NOTE` прописаны как глобальные переменные окружения Jinja2 — то есть доступны *любому* шаблону без передачи из маршрута. Заккрытие служебной информации в продукте начинается с удаления этих глобалов и ревизии шаблонов.

9. Пользовательские сценарии

9.1. Кандидат: от входа до принятого приглашения

1. **Вход.** `/login?role=candidate` → email → код из «письма» (в демо — на экране) → `/verify`.
2. **Шаг 1 — профиль и барометр.** ФИО, город, специализация, стек, желаемая роль, формат работы, согласие на обработку данных. Система сразу считает флаги барометра и предварительный грейд.
3. **Шаг 2 — история ФСП.** Подключение реестра: показываются достижения, сумма баллов, категория S-D и старшая роль. Можно пропустить — тогда `fsp_score = 0`, категория D.
4. **Шаг 3 — сертификат КИТ.** Загрузка PDF/TXT: грейд, баллы, специальность, номер и дата. Грейд из сертификата перезаписывает предварительный и даёт метку «верифицирован». Можно пропустить и загрузить позже в кабинете.
5. **Кабинет.** Кандидат видит: итоговый грейд и его источник, вердикт барометра, баллы и категорию ФСП, данные сертификата, строку матрицы видимости («топ ФСП», «опытный специалист», «новичок с амбициями» ...), предупреждения по профилю, входящие приглашения. Здесь же — переключатель видимости в банке, редактирование анкеты, замена/удаление сертификата.
6. **Банк потребностей.** `/needs` — список открытых вакансий с обязательной вилкой; `/needs/{id}` — описание, рейтинг компании словами и цифрой, предупреждения о сомнительных формулировках, форма отклика.
7. **Отклик.** Сообщение → статус `sent`; работодатель переводит его в `viewed` / `accepted` / `rejected`.
8. **Приглашение.** Кандидат видит сообщение, вилку, уровень процессов компании *словами* и её рейтинг. Принять → работодатель получает ФИО и email; отклонить → контакты не раскрываются.

9.2. Работодатель: от анкеты до раскрытия контактов

1. **Вход.** `/login?role=employer` → email → код.
2. **Шаг 1 — профиль компании.** Название, город, контактное лицо, должность, описание, согласие.
3. **Шаг 2 — анкета процессов.** Шесть нейтральных вопросов; ответы сохраняются в JSON, из них считается тир. В демо-сборке уровень показывается словами сразу после сохранения (в продукте — нет, см. главу 10, п. 5).
4. **Потребность.** `/employer/vacancies/new`: специализация, грейд и режим сопоставления (строго / ± 1 / не важен), стек, вилка (обязательна), формат, город, размер команды, описание.
5. **Подборка.** Страница потребности показывает ранжированный список: сначала фильтр по матрице видимости и полноте профиля, затем отбор по специализации, затем сортировка по релевантности и правилам раздела 6.6. Отдельно — число и замаскированные имена закрытых профилей.
6. **Банк кандидатов.** `/employer/bank` с фильтром по специализации; карточки анонимны: маскированное имя, специализация, грейд, метка верификации, баллы КИТ, стек, достижения ФСП; для тира 2 добавляются амбициозность и роль ФСП, для тира 3 — город и полный профиль.
7. **Приглашение.** Сообщение + вилка → `sent`. Статусы видны в `/employer/invitations`.
8. **Отклики.** `/employer/applications`: новые сверху, далее по убыванию релевантности; смена статуса одним действием.
9. **Раскрытие контактов.** Только после `accepted` со стороны кандидата работодатель видит ФИО и email — до этого в карточке маскированное имя, а в приглашении соответствующая сноска.

Сквозной демо-сценарий для защиты проекта

1) Зарегистрировать кандидата из мок-реестра ФСП (топ-уровень) и загрузить демо-сертификат КИТ → профиль становится «топ ФСП». 2) Зарегистрировать двух работодателей: с 6 «здоровыми» ответами (тир 3) и с 2 (тир 1). 3) Создать потребность у второго → топ-кандидат попадёт в закрытые профили, у первого → окажется на верхней строке подборки. 4) Отправить приглашение, принять его кандидатом → контакты раскрываются только у работодателя тира 3, пригласившего кандидата.

10. Демонстрационный режим: что видно сейчас и что закрывается в продукте

Главное, что нужно знать о текущей сборке

Сейчас платформа намеренно работает в режиме **полной демонстрационной прозрачности**: в интерфейсе видны все внутренние логики и административные процессы — как начисляются баллы и релевантность, почему профиль закрыт, какой у работодателя уровень процессов, откуда берутся данные ФСП и рейтинг компании, какой код подтверждения и какие учётные записи тестовые.

Это сделано для защиты проекта: жюри и заказчик должны видеть, что ранжирование детерминированно, объяснимо и соответствует ТЗ, а не является «чёрным ящиком». **В продуктивной версии кандидат и работодатель этого видеть не будут.** Ниже — полный перечень с обоснованием по каждому пункту.

10.1. Алгоритмы, веса и начисление баллов

Что видно сейчас	Где именно	Почему в продукте закрыто	Что остаётся пользователю
1. Веса релевантности 35 / 25 / 20 / 10 / 10 (MATCH_WEIGHTS)	Работодателю: страница потребности /employer/vacancies/{id} (vacancy_detail.html , 53–55 и 125), карточка соответствия _partial_match.html (18–22). Переменная глобальна для всех шаблонов (web.py)	Формула ранжирования — ядро продукта и коммерческая тайна. Публикация весов позволяет «заточить» профиль и текст вакансии под алгоритм (гейминг), обесценивает подборку и создаёт почву для споров о дискриминации: видно, что неverified грейд штрафует, а амбициозность даёт бонус	Процент релевантности и 2–3 человеческие причины без чисел: «стек совпал на 3 из 5», «грейд подтверждён сертификатом КИТ», «история ФСП: категория А»
2. Поштучное начисление баллов — список reasons из score_match(), включая штраф verified // 3 и бонус +1 за амбициозность	Работодателю: _partial_match.html на дашборде, в банке, в подборке потребности и в откликах	Раскрывает внутреннюю методику целиком, в том числе штрафные ветки; позволяет восстановить алгоритм по одному экрану и манипулировать им	Краткое объяснение выбора системы («почему этот кандидат выше») без арифметики и без указания штрафов
3. Методика баллов ФСП: сумма points по достижениям и пороги категорий 150 / 100 / 60 / 30 → S/A/B/C/D	Кандидату: /candidate (candidate_home.html , 81–84), шаг 2 регистрации /register/candidate/fsp ; категории — в карточках работодателя	Методика начисления баллов принадлежит оператору ФСП: платформа должна получать готовые баллы и категорию из реестра, а не считать их сама. Публикация порогов провоцирует накрутку достижений и споры о «несправедливой» категории	Итог: «Баллы и категория получены из реестра ФСП, дата синхронизации ...» + список самих достижений (место, роль, соревнование)
4. Баллы за каждое достижение (points в мок-реестре, таблица achievement)	Кандидату: шаг 2 регистрации и кабинет; таблица достижений с колонкой баллов	В MVP значения тестовые и не подтверждены оператором ФСП; показывать «официальные» баллы из мока нельзя — это введение пользователя в заблуждение	Достижения без весов; баллы — только как агрегат из официального реестра

10.2. Разграничение доступа: тир, матрица, причины закрытия

Что видно сейчас	Где именно	Почему в продукте закрыто	Что остаётся пользователю
5. Уровень процессов работодателя словами (<code>tier_label</code>) и его расшифровка (<code>TIER_DESCRIPTIONS</code>)	Работодателю: <code>/employer</code> (<code>employer_home.html</code> , 16 и 167), <code>/employer/bank</code> (13, 79), <code>/employer/vacancies/{id}</code> (130–131), <code>/register/employer/survey</code> (14)	Прямое нарушение ТЗ 5.2: «работодатель не видит свой тир в интерфейсе. Ни цифрой, ни словами». Зная уровень и его расшифровку, компания понимает, какие ответы анкеты «повышают» доступ, — анкета теряет смысл и превращается в тест на угадывание	Работодатель видит только результат: размер и состав подборки. Максимум — нейтральная подсказка «анкета процессов влияет на доступность профилей»
6. Матрица видимости целиком — все три уровня с описаниями и условиями доступа	Анонимно на лендинге <code>/</code> (<code>index.html</code> , 47–53) и кандидату в кабинете (<code>candidate_home.html</code> , 236–241)	Раскрывает механизм сегментации рынка труда: любой работодатель заранее знает, какие ответы дают доступ к «топ ФСП». Кандидату по ТЗ положен только словесный уровень конкретной компании в приглашении, а не вся матрица	Кандидату — собственная строка матрицы («ваш профиль виден организациям с развитыми процессами») и уровень конкретной компании в приглашении; полная матрица — во внутренней документации и правилах сервиса
7. Причина, почему профиль закрыт (<code>restricted_reason</code> , <code>visibility_note</code>): «профиль заполнен не полностью: ...», «кандидат скрыл профиль из банка», «уровень профиля выше, чем доступен при текущем уровне организации процессов»	Работодателю: банк (счётчик закрытых + замаскированные имена), страница потребности (<code>restricted_names</code>), карточка закрытого профиля <code>employer_candidate_restricted.html</code> , <code>_partial_candidate_card.html</code> (49)	Третья формулировка фактически сообщает работодателю его собственный тир (обход запрета ТЗ 5.2) и раскрывает существование конкретных скрытых кандидатов. Причина «уровень профиля выше» читается как оценка компании и несёт репутационные и юридические риски (споры о дискриминации, претензии ФАС и трудовых инспекций)	Нейтральное «Профиль недоступен» + агрегированный счётчик; подробности — только по обращению в службу поддержки. Замаскированные имена недоступных кандидатов не показываются
8. Перечень закрытых полей (<code>TIER_FIELDS</code> → <code>visible_fields</code>): «Вам доступны поля: ...»	Работодателю: <code>/employer</code> , <code>/employer/bank</code> (81), <code>/employer/vacancies/{id}</code> (111, 133)	Показывает не только то, что открыто, но и чего работодатель лишён и как это «открыть» — то есть снова раскрывает тир и стимул манипулировать анкетой	Показываются только реально открытые поля, без перечисления скрытых и без сравнения с другими уровнями

10.3. Демо-данные, аутентификация и техническая поверхность

Что видно сейчас	Где именно	Почему в продукте закрыто	Что остаётся пользователю
9. Код подтверждения email прямо на экране (demo_code) и сноска AUTH_DEMO_NOTE	Всем, кто ввёл email: /verify (verify.html , 31), сноска в login.html (40) и в подвале каждой страницы (base.html , 64); код передаётся из маршрута routers/auth.py:73	Это полный обход аутентификации: любой, кто знает чужой email, входит в чужой кабинет. SMTP не подключён — заглушка существует только для демонстрации	«Код отправлен на вашу почту» (или вход через ФСП ID / Keycloak / magic link). Код не отображается, не логируется и не возвращается в ответе
10. Демо-учётные записи и мок-реестр ФСП: 8 email вида *@fsp.demo , 9 учётных записей демо-стенда seed.py (6 кандидатов и 3 компании), 4 «демо-компании» с заданными сигналами, подсказки логинов на форме входа	Анонимно: /login (login.html , 44–45 — chips artem.petrov@, anna.sokolova@; блок «Демо-вход» печатает учётные записи демо-стенда из seed.demo_accounts()), /register/candidate/fsp (routers/registration.py:67 — demo_emails=sorted(fsp_mock.FSP_REGISTRY)); данные — fsp_mock.py , employer_rating.RATING MOCK , seed.py	Публикация тестовых учёток и состава «истории ФСП» позволяет любому зайти под чужим демо-профилем и увидеть чужие достижения; в продукте данные приходят из официального реестра, а подсказки недопустимы	Регистрация только по собственному email; история ФСП подтягивается из реестра по подтверждённой учётной записи
11. Демо-сертификат КИТ и сырой текст распознавания (DEMO_CERTIFICATE_TEXT , kit_raw , kit_source)	Кандидату: шаг 3 регистрации и кабинет; kit_block["raw"] в helpers.py кладёт распознанный текст в контекст шаблонов	Сырой текст распознавания — отладочная информация: по нему восстанавливаются правила парсера и подделывается «пройденная» проверка. Демо-сертификат позволяет получить верифицированный грейд без реальной проверки	Только извлечённые поля (грейд, баллы, номер, дата) и статус проверки; при ошибке распознавания — предложение ввести данные вручную

Что видно сейчас	Где именно	Почему в продукте закрыто	Что остаётся пользователю
12. Открытая техническая поверхность: Swagger UI <code>/api/docs</code> , OpenAPI-схема, <code>/health</code> , файл <code>database.db</code> в корне репозитория, путь к БД через <code>DATABASE_URL</code>	Всем без авторизации: <code>main.py</code> (<code>docs_url="/api/docs"</code> , маршрут <code>/health</code>); SQLite-файл с данными лежит рядом с кодом	Публичная карта всех маршрутов и параметров облегчает подбор эндпоинтов и атаки; файл БД содержит персональные данные (ФИО, email, достижения) — его распространение нарушает 152-ФЗ; <code>/health</code> не должен раскрывать сведения о приложении	Swagger отключён (<code>docs_url=None</code>) или доступен только во внутренней сети; <code>/health</code> — минимум данных для мониторинга; БД — на защищённом томе с резервными копиями, в репозитории её нет

10.4. Рейтинг компании, модерация и барометр амбиций

Что видно сейчас	Где именно	Почему в продукте закрыто	Что остаётся пользователю
13. Рейтинг компании: сырые сигналы и арифметика — число тендеров, наличие в реестре отечественного ПО, средняя оценка сотрудников, разрыв зарплат топ-менеджмента и инженеров, коэффициенты формулы, сноска <code>RATING_DEMO_NOTE</code>	Кандидату: карточка потребности (<code>_partial_vacancy_card.html</code> , 44), <code>/needs/{id}</code> (<code>need_detail.html</code> , 49), приглашение, <code>register_employer_profile.html</code> (82), лендинг (75); расчёт — <code>employer_rating.company_rating()</code>	В MVP сигналы — заглушка (для недемо-компаний детерминированно из MD5-хеши названия), то есть цифра не отражает реальность. В продукте сигналы парсятся из открытых источников, а методика становится внутренним регламентом: её публикация ведёт к накрутке отзывов и тендеров и к юридическим рискам — оценка компании с указанием «разрыва зарплат» без верифицированных источников может быть оспорена как недостоверная информация	Итоговая метка («высокий / средний / низкий уровень») и, при необходимости, ссылки на первоисточники. Объяснение — по запросу в поддержку, без коэффициентов
14. Словарь «красных флагов» — 13 триггерных формулировок (<code>RED_FLAG_MARKERS</code>) и список замечаний к профилю (<code>red_flags()</code>)	Кандидату: страница потребности и кабинет (<code>candidate_home.html</code> , 65–68); работодатель видит отметки в описании своей потребности	Публикация словаря модерации позволяет обойти фильтр переформулировками; показ «замечаний к своему профилю» в текущих формулировках дублирует внутренние правила ранжирования (грейд не подтверждён, 0 баллов ФСП, меньше двух флагов барометра)	Факт проверки: «вакансия проверена модератором» / «требуется уточнения в профиле» и конкретное действие для пользователя (загрузить сертификат, заполнить поле) — без списка триггеров

Что видно сейчас	Где именно	Почему в продукте закрыто	Что остаётся пользователю
15. Барометр амбиций: флаги и порог — 4 формулировки (<code>AMBITIONS</code>), текст «Барометр амбиций: N из 4 флагов (порог — 2)...»	Анонимно на лендинге (<code>index.html</code> , 31–37), кандидату на шаге 1 регистрации и в кабинете (<code>ambitions_note</code>)	Раскрывает, какие именно ответы делают профиль «амбициозным», а это влияет на видимость работодателям (строка матрицы «новичок с амбициями» требует тир 2). Пользователь начинает отвечать не честно, а «на нужный результат» — анкета теряет смысл, как и анкета работодателя	Вопросы без подсказок о пороге; кандидату — только вердикт («амбициозный профиль») и возможность его уточнить

10.5. Административные процессы и эксплуатация

Как устроено сейчас	Где именно	Почему в продукте иначе	Что появляется в продукте
16. Администрирование = доступ к коду и файлу БД. Ролей «администратор» и «модератор» нет; таблицы создаются <code>SQLModel.metadata.create_all</code> ; данные наполняются моками; правки — вручную через SQLite	<code>database.py</code> (<code>init_db</code>), <code>fsp_mock.py</code> , <code>employer_rating.py</code> , файл <code>database.db</code> в корне проекта	В продукте любая правка данных должна быть учтена и обоснована: без журнала действий невозможно расследовать инцидент, ответить на претензию кандидата или подтвердить корректность ранжирования. Ручные правки БД недопустимы	Админ-панель с ролями, журнал аудита (кто, что, когда изменил), миграции схемы (Alembic), резервные копии, разделение dev / stage / prod-окружений
17. Служебные поля и внутренняя механика в контексте страниц: <code>fsp_demo</code> , <code>kit_source</code> , <code>specialization_manual</code> , <code>grade_override</code> , <code>is_new</code> , <code>last_visit_at</code> , <code>consent</code> , <code>visible_in_bank</code> ; flash base64-JSON в cookie; редиректы 303	<code>web.py</code> (<code>render()</code> , <code>flash()</code>), <code>helpers.py</code> , шаблоны; видно в DevTools и в исходном коде страницы	Само по себе не секрет, но в продукте служебные данные не должны попадать в HTML и логи: по ним восстанавливается внутреннее устройство, а лишние поля в контексте — типичная причина утечек (в карточке кандидата уже лежат <code>real_name</code> и <code>email</code>)	Минимальный контекст шаблонов (DTO вместо «всего объекта»), отдельные view-модели без служебных полей, cookie с флагами <code>Secure</code> и <code>SameSite</code> , фильтрация чувствительных полей из логов
18. Нет продуктивных средств защиты и сопровождения: без ограничения частоты запросов и капчи на <code>/login</code> и <code>/verify</code> , без лимитов приглашений и откликов, без модерации текстов, без email-уведомлений, без хранения текста согласия 152-ФЗ (только <code>consent: bool</code>), без механизма удаления профиля по требованию	Отсутствует в коде: <code>routers/auth.py</code> , <code>routers/employer.py</code> , <code>routers/candidate.py</code>	Шестизначный код с TTL 15 минут без ограничения попыток перебирается; массовая рассылка приглашений превращает платформу в спам; согласие на обработку ПДн должно храниться как артефакт (версия текста, дата, IP), а у пользователя должно быть право на удаление данных	Rate limit и блокировка после N попыток, капча, суточные лимиты действий, премодерация вакансий и сообщений, email-уведомления, реестр согласий, процедура удаления и обезличивания профиля, мониторинг и алерты

10.6. Почему это закрывается: пять причин

- 1. Коммерческая тайна и защита от гейминга.** Веса, пороги и коэффициенты — ядро продукта. Как только формула публична, пользователи оптимизируют анкеты под неё, а не отражают реальность: подборка

теряет ценность.

2. **Персональные данные (152-ФЗ).** ФИО, email, город, достижения и распознанный текст сертификата — ПДн. Демонстрационные механизмы (код на экране, демо-учётки, файл БД в репозитории, сырой текст распознавания) в продукте недопустимы: нужны минимизация данных, защищённое хранение, реестр согласий и право на удаление.
3. **Юридические и репутационные риски.** Сообщения вида «уровень профиля выше, чем доступен при текущем уровне организации процессов» и публичный рейтинг с «разрывом зарплат» — оценочные суждения о компании и о кандидате. Без верифицированных источников и закрытой методики это повод для претензий и судов.
4. **Требование Т3.** Раздел 5.2 прямо запрещает показывать работодателю его тир — ни цифрой, ни словами; категории ФСП и рейтинг предназначены только кандидату. Демонстрационная сборка эти границы намеренно раздвигает, и это надо закрывать до продуктивного релиза.
5. **Безопасность.** Открытый Swagger, показ кода подтверждения, отсутствие rate limit и модерации — демонстрационные удобства, которые в продукте становятся векторами атаки и спама.

Что сознательно остаётся открытым и в продукте

Прозрачность правил *для пользователя* сохраняется там, где она полезна и безопасна: обязательная вилка зарплаты; факт верификации грейда сертификатом КИТ; состав открытых полей карточки; словесный уровень процессов компании в приглашении кандидата; статусы приглашений и откликов; правило раскрытия контактов только после принятия приглашения; возможность скрыть свой профиль из банка; объяснение релевантности в человеческих формулировках — но без весов, порогов и коэффициентов.

11. Известные отличия реализации от ТЗ и ограничения MVP

11.1. Отличия от ТЗ

№	Отличие	Пояснение и влияние
1	<code>Employer.tier</code> хранится в БД	ТЗ 5.2 требует не хранить тир и считать его на лету. В реализации значение сохраняется при каждом сохранении анкеты (<code>services.apply_employer_survey</code>) и пересчитывается из <code>survey_answers</code> , поэтому расхождения с ответами не возникает, но формально поле в схеме есть.
2	Работодатель видит свой уровень процессов словами	Отступление от ТЗ 5.2 сделано сознательно для демонстрации — закрывается в продукте (глава 10, п. 5).
3	Имена полей вакансии	ТЗ: <code>salary_from / salary_to</code> , <code>is_published</code> . Реализация: <code>salary_min / salary_max</code> , <code>open</code> . Добавлены <code>grade_match</code> , <code>city</code> , <code>team_size</code> , <code>company</code> , <code>is_new</code> .
4	Таблица <code>achievement</code> не является источником данных	Достижения берутся из мок-реестра по email (<code>fsp_mock.get_achievements</code>). Таблица создана по ТЗ 5.4 и готова к заполнению при подключении реестра ФСП.
5	Двусторонняя механика	Кроме приглашений работодателя реализованы банк потребностей <code>/needs</code> и отклики кандидата (<code>Application</code>) — расширение разделов 5.6 и 10.3 ТЗ.
6	Список специализаций шире	Добавлены <code>mobile</code> и <code>embedded</code> (робототехника/БАС) сверх перечня ТЗ — под дисциплины ФСП.
7	Нет Docker	ТЗ 3 предполагает «один <code>docker-compose up</code> ». В репозитории нет <code>Dockerfile</code> и <code>compose</code> -файла; запуск — через <code>uvicorn</code> (глава 12).
8	Тесты — только смоук-набор	В <code>tests/</code> перенесены три smoke-сценария (63 проверки <code>pytest</code> : маршруты и <code>guard</code> -функции, строгий рендер шаблонов, сквозной сценарий работодателя). Юнит-покрытия <code>logic.py</code> (формулы релевантности, матрицы видимости, грейдов) и CI нет — это пункт P2 главы 13.
9	Стили	Используется локальный <code>pico.min.css</code> + собственный <code>app.css</code> . Каталог <code>tailwindcss/</code> и файл <code>static/css/TZ</code> — неиспользуемые артефакты (49 и 0 байт).
10	Время и сессии	Метки времени — строки ISO-8601 UTC, а не тип <code>datetime</code> . Сессии хранятся в таблице БД, а не в JWT; код подтверждения не хешируется.
11	Верификация как свойство	В ТЗ <code>is_verified</code> — поле; в реализации — производное свойство из <code>kit_connected</code> и данных сертификата, что исключает рассинхронизацию.
12	Рейтинг недемо-компаний	Для компаний вне списка <code>RATING MOCK</code> сигналы детерминированно вычисляются из MD5-хеши названия: повторяемо, но содержательно бессмысленно — до подключения внешних источников.

11.2. Ограничения MVP

- Нет уведомлений (email, пуш): о приглашении и отклике пользователь узнаёт только в кабинете; счётчик «новых» считается по `last_visit_at`.
- Нет чата и вложений: коммуникация — одно сообщение в приглашении/отклике.
- Нет поиска по банку кроме фильтра по специализации; нет пагинации — записи выбираются в память целиком (`list_candidates` и др.).
- Нет модерации текстов, жалоб, блокировок пользователей.
- Нет логирования, метрик, экспорта данных, интернационализации.
- SQLite в одном файле с `check_same_thread=False`: приемлемо для сотен записей, не рассчитано на высокую параллельную нагрузку.

12. Запуск, проверка и демо-данные

12.1. Установка и запуск

```
# 1. Окружение (Python 3.13)
python3.13 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# 2. Запуск: таблицы создаются при старте, пустая база заполняется демо-стендом
uvicorn main:app --reload
#   → http://127.0.0.1:8000

# 3. Необязательно: другой файл/путь БД
export DATABASE_URL=sqlite:///database.db

# 4. Демо-стенд вручную: отчёт о составе либо полная пересборка
python3.13 seed.py
python3.13 seed.py --force
```

Проверка готовности: `GET /health` → `{"status":"ok","app":"ФСП «спец. трек»"}`. Документация API в демо-сборке открыта на `/api/docs` (в продукте закрывается — глава 10, п. 12).

12.2. Демо-вход и демо-данные

Что	Как получить в демо-сборке
Вход	Любой корректный email и роль; код подтверждения показывается на странице <code>/verify</code> . SMTP не подключён.
История ФСП	Email из мок-реестра <code>fsp_mock.FSP_REGISTRY</code> : <code>artem.petrov@ · anna.sokolova@ · mark.ivanov@ · olga.smirnova@ · sergey.orlov@ · maria.lebedeva@ · nikita.zaitsev@ · sofia.morozova@</code> (домен <code>fsp.demo</code>). Любой другой email — сценарий «истории ФСП нет»: 0 баллов, категория D.
Сертификат КИТ	Сохранить текст <code>kit_parser.DEMO_CERTIFICATE_TEXT</code> в файл <code>kit.txt</code> и загрузить на шаге 3 регистрации или в кабинете (<code>POST /candidate/kit</code>). Распознаются грейд Senior, 84 балла, специальность «техническая поддержка» → support.
Демо-компании рейтинга	ТехКорп, Вектор Софт, ПромТех Групп, ОлдЛайн — сигналы заданы явно в <code>employer_rating.RATING MOCK</code> .
Демо-стенд (<code>seed.py</code>)	Заполняет пустую базу один раз при старте приложения (<code>lifespan</code> → <code>seed.ensure_demo_data</code>): 6 кандидатов (топ ФСП, крепкий мидл, амбициозный новичок, новичок без амбиций, кандидат без истории ФСП, junior без сертификата), 3 работодателя тиров 3/2/1, 4 потребности и 2 приглашения (<code>accepted</code> и <code>sent</code>). Анкеты создаются штатными сервисами, поэтому грейд, специализация, амбициозность, категория ФСП и тип рассчитываются правилами приложения, а не записываются вручную. Пересборка стенда — <code>python3.13 seed.py --force</code> . Доступы: <code>artem.petrov@ · anna.sokolova@ · mark.ivanov@ · maria.lebedeva@</code> (<code>fsp.demo</code>), <code>egor.timofeev@ · pavel.kulagin@</code> (<code>demo.local</code>), <code>hr@progress-soft.demo · hr@stabilit.demo · hr@fast-dev.demo</code> — этот же список выводится на <code>/login</code> из <code>seed.demo_accounts()</code> .
Состояние БД на момент черновика	<code>database.db</code> заполнен демо-стендом: 6 кандидатов, 3 работодателя, 4 потребности, 2 приглашения, 0 откликов и 0 сессий. Пересобрать стенд — <code>python3.13 seed.py --force</code> ; если файл удалить, таблицы и демо-набор создадутся заново при следующем старте приложения.

12.3. Проверки, выполненные перед выпуском черновика

Проверка	Результат	Что гарантирует
Компиляция всех модулей Python	ОК	Нет синтаксических ошибок и битых импортов.
Компиляция шаблонов Jinja2	30 шаблонов	Все шаблоны парсятся, наследование корректно.

Проверка	Результат	Что гарантирует
Обход GET-маршрутов	24 пути / 72 запроса	Каждый путь проверен для анонима, кандидата и работодателя: нет 500, доступ закрыт редиректом 303.
Рендер всех страниц при <code>StrictUndefined</code>	OK	В шаблонах не осталось необъявленных переменных — «тихий» пропуск контекста нет.
Изоляция ролей	OK	Кандидат не получает 200 на <code>/employer/*</code> , работодатель — на <code>/candidate/*</code> и <code>/needs/*</code> ; аноним на приватных путях редиректится на <code>/login</code> .
Сквозной сценарий работодателя	20 проверок	Анкета → тир → банк → потребность → приглашение → смена статусов → раскрытие контактов проходят без ошибок.
Демо-стенд (<code>tests/test_seed.py</code>)	17 проверок	Состав стенда, матрица видимости по тирам (тир 3 → 6 анкет, тир 2 → 5, тир 1 → 2), тир из ответов анкеты, ранжирование под потребности, идемпотентность сйда и пересборка <code>--force</code> .
Специализация по КИТ (<code>tests/test_kit_specialization.py</code>)	15 проверок	Специализация определяется по полю специальности сертификата, а не по всему тексту документа: шапка «Прошел(а) тестирование ...» больше не уводит анкету в QA; приоритет КИТ → ФСП → стек → ручной выбор сохранён.
Итог <code>pytest</code>	95 passed	Весь набор <code>tests/</code> зелёный на Python 3.13; каждый модуль проходит и по отдельности.

Проверки перенесены из разовых smoke-скриптов в каталог `tests/` и запускаются через `pytest`. Тесты работают на временной копии базы (`DATABASE_URL` выставляется в `tests/conftest.py` до импорта приложения), поэтому `database.db` из репозитория не изменяется — состояние стенда, описанное в п. 12.2, остаётся прежним.

12.4. Как повторить проверки

Команды предполагают активированное окружение `.venv`. Системный `python3` зависимостей не содержит — поэтому явно указан `python3.13` (если его нет в PATH, подставьте полный путь к интерпретатору).

```
# компиляция модулей
python3.13 -m compileall -q .

# весь смоук-набор (testpaths = tests задан в pytest.ini)
python3.13 -m pytest

# отдельные сценарии
python3.13 -m pytest tests/test_smoke_routes.py -v      # маршруты, шаблоны, guard-функции
python3.13 -m pytest tests/test_smoke_strict.py -v     # строгий рендер всех страниц
python3.13 -m pytest tests/test_smoke_employer_flow.py -v # сквозной сценарий работодателя
python3.13 -m pytest tests/test_seed.py -v             # демо-стенд: состав, видимость,
ранжирование                                           #
python3.13 -m pytest tests/test_kit_specialization.py -v # специализация по сертификату КИТ
python3.13 -m pytest -k invite                         # только приглашения и контакты

# демо-стенд: отчёт о составе и пересборка
python3.13 seed.py
python3.13 seed.py --force
```

12.5. Пересборка этого PDF

```
bash docs/build_pdf.sh      # documentation.html → FSP-documentation.pdf (headless Chrome)
```

13. Чек-лист вывода в продакшн

Приоритет	Задача	Где менять	Критерий приёмки
P0	Убрать показ кода подтверждения; подключить SMTP либо ФСР ID / Keycloak	<code>routers/auth.py</code> (передача <code>demo_code</code>), <code>verify.html</code> , <code>content.AUTH_DEMO_NOTE</code>	Код не встречается в HTML, ответах и логах; вход возможен только через почту/ФСР ID
P0	Скрыть тип от работодателя (требование ТЗ 5.2)	<code>routers/employer.py</code> ; шаблоны <code>employer_home.html</code> , <code>employer_bank.html</code> , <code>vacancy_detail.html</code> , <code>register_employer_survey.html</code> , <code>employer_candidate_restricted.html</code>	Ни цифры, ни словесного уровня, ни <code>TIER_DESCRIPTIONS</code> в кабинете работодателя
P0	Заменить причины закрытия профиля нейтральными; убрать замаскированные имена недоступных кандидатов	<code>logic.restricted_reason</code> , <code>candidate_visible_profile</code> , <code>helpers.bank_rows</code> , <code>helpers.vacancy_candidates</code> , шаблоны карточек	Работодатель видит «Профиль недоступен» и агрегированный счётчик, без указания уровня процессов
P0	Убрать веса и арифметику релевантности из интерфейса	<code>web.py</code> (глобал <code>MATCH_WEIGHTS</code>), <code>vacancy_detail.html</code> , <code>_partial_match.html</code>	Пользователь видит процент и человеческие причины; числа весов и штрафов отсутствуют
P0	Закрыть техническую поверхность: Swagger, <code>/health</code> , файл БД в репозитории, демо-учётки и подсказки логинов	<code>main.py</code> (<code>docs_url</code>), <code>login.html</code> , <code>routers/registration.py</code> (<code>demo_emails</code>), <code>.gitignore</code> (создан: кеш Python, <code>.venv</code> , <code>.pytest_cache</code> ; <code>database.db</code> пока не исключён)	<code>/api/docs</code> недоступен извне; <code>database.db</code> не в репозитории; подсказок демо-логинов нет
P0	Rate limit и блокировка после N попыток на <code>/login</code> и <code>/verify</code> ; хеширование кода	<code>auth.py</code> , <code>routers/auth.py</code>	Перебор кода невозможен; в БД хранится хеш кода, а не значение
P1	Подключить реестр ФСР вместо мока; писать достижения в таблицу <code>achievement</code>	<code>fsp_mock.py</code> , <code>services.connect_fsp</code> , <code>helpers.fsp_block</code>	Баллы и категория приходят из реестра; <code>FSP_DEMO_NOTE</code> удалена
P1	Подключить внешние источники рейтинга и закрыть методику	<code>employer_rating.py</code> , <code>_partial_vacancy_card.html</code> , <code>need_detail.html</code>	Сигналы берутся из верифицированных источников; пользователю показана только метка
P1	Верификация сертификата КИТ через API центра оценки; не показывать <code>kit_raw</code>	<code>kit_parser.py</code> , <code>services.apply_kit</code> , <code>helpers.kit_block</code>	Грейд подтверждается запросом к провайдеру; сырой текст распознавания не в интерфейсе
P1	Cookie <code>Secure</code> + HTTPS, хранение согласий 152-ФЗ как артефакта, процедура удаления профиля	<code>web.py</code> (<code>set_cookie</code>), <code>models.py</code> , <code>routers/candidate.py</code>	Сессия только по HTTPS; есть версия текста согласия, дата и IP; удаление профиля работает
P1	Журнал аудита и роль модератора; премодерация вакансий и сообщений	новый модуль + <code>routers/employer.py</code>	Действия пользователей и модераторов логируются; публикация только после проверки

Приоритет	Задача	Где менять	Критерий приёмки
P2	Миграции схемы (Alembic), Dockerfile и docker-compose (требование ТЗ 3), бэкапы, мониторинг	корень репозитория, <code>database.py</code>	Подъём стенда одной командой; схема меняется миграциями; есть копии и алерты
P2	CI и юнит-покрытие <code>logic.py</code> (смоук-набор уже перенесён в <code>tests/</code>)	CI-конфигурация, <code>tests/</code>	<code>pytest</code> прогоняется в CI на каждый коммит; формулы релевантности, матрица видимости и правила грейдов покрыты юнит-тестами
P2	Пагинация банка и подборок, суточные лимиты приглашений и откликов, уведомления на email	<code>helpers.py</code> , <code>routers/employer.py</code> , <code>routers/candidate.py</code>	Списки постраничные; лимиты действуют; пользователь получает уведомления

Порядок работ: сначала P0 (безопасность, персональные данные и закрытие служебной информации), затем интеграции P1, затем инфраструктура P2. Пункты P0 соответствуют пунктам 1–12 главы 10.

14. Приложение: справочник констант

Константа	Где определена	Значение и назначение
MATCH_WEIGHTS	logic.py	specialization 35, stack 25, grade 20, verified 10, fsp 10 — веса релевантности (сумма 100).
MATCH_FSP_POINTS	logic.py	S 10, A 9, B 7, C 5, D 2 — баллы категории ФСП внутри критерия «история ФСП».
AMBITIONS, AMBITION_THRESHOLD	logic.py	4 флага барометра; порог вердикта «амбициозный профиль» — 2.
TIER_LABELS	logic.py	3 — «развитые процессы», 2 — «базовые процессы», 1 — «начальные процессы» (формулировки для кандидата).
TIER_DESCRIPTIONS	logic.py	Расшифровка уровней с указанием доступности банка; в демо видна всем — закрывается в продукте.
TIER_FIELDS	logic.py	Состав полей карточки кандидата для тиров 1 / 2 / 3.
TIER_QUESTIONS	logic.py	Порядок полей анкеты: q1_lead_time ... q6_benefits; tier_score = число истинных ответов.
GRADE_MATCH_MODES, DEFAULT_GRADE_MATCH_MODE	logic.py	exact / plus_minus / any; по умолчанию plus_minus.
PROFILE_REQUIRED_FIELDS	logic.py	Имя, специализация, роль в команде, формат работы; плюс согласие 152-ФЗ.
RED_FLAG_MARKERS	logic.py	13 маркеров сомнительных формулировок вакансии.
CONTACT_REVEAL_NOTE	logic.py	Единственное правило раскрытия ФИО и контактов — после принятия приглашения.
FSP_CATEGORY_THRESHOLDS	fsp_mock.py	150 → S, 100 → A, 60 → B, 30 → C, иначе D.
FSP_REGISTRY	fsp_mock.py	8 тестовых участников с достижениями; источник баллов ФСП в MVP.
RATING MOCK	employer_rating.py	Сигналы 4 демо-компаний: тендеры, реестр ПО, отзывы, разрыв зарплат.
DEMO_CERTIFICATE_TEXT	kit_parser.py	Пример сертификата КИТ из раздела 8.1 ТЗ — для демо-загрузки и проверок.
SESSION_TTL_DAYS, CODE_TTL_MINUTES	web.py	7 дней и 15 минут — сроки жизни сессии и кода подтверждения.
SESSION_COOKIE, FLASH_COOKIE	web.py	fsp_session, fsp_flash.
FSP_DEMO_NOTE, AUTH_DEMO_NOTE, RATING_DEMO_NOTE	content.py	Сноски честности о мок-данных; в продукте удаляются вместе с моками.
EMPLOYER_SURVEY, CANDIDATE_BAROMETER	content.py	Формулировки вопросов анкет — нейтральные, без подсказки «правильного» ответа.
SPECIALIZATIONS, GRADES, TEAM_ROLES, FSP_ROLES, WORK_FORMATS, VACANCY_FORMATS, FSP_CATEGORY_TITLES	logic.py	Справочники для форм и карточек.

ФСП «спец. трек» · черновик документации v0.9 · октябрь 2026 г. Документ описывает текущую демонстрационную сборку: перечень расхождений с продуктовым поведением — глава 10, план закрытия — глава 13. Источник требований: TZ.txt и «ФСП спец. трек ТЗ.pdf» в корне репозитория.